ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 31/10/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:

https://github.com/192524025simats-collab/DSA0116_192524025/tree/main/ASSIGNMENT/CO2

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.
Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Bloom's Level: K5 – Evaluate

Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to students' fees. A function written to apply the concession appears to work when its result is printed inside the function, but the fee stored for the student never changes.

Your Challenge

Write the concession function three ways – passing the fee by value, by reference and by pointer. Before running, predict the stored fee after each call, then verify.

Inquiry

Explain where each version stores its parameter, why only two of them change the caller's fee, and when a const reference parameter should be used instead.

Design Decision

Several concessions may apply to one student. Decide whether the function should change the fee in place or return the new fee, and defend your choice.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

void change(int a, int& b, int* c)

{
```

```
    a += 10;
    b += 10;
    *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

My predicted output (write before running):

Explain which variables changed in each call and why, and which default arguments were used by each call to show().

Complete the Code

Complete TODO 1 below so that the caller's fee is changed, and execute the program.

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100; // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
```

```
    // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by reference)
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:

after call by value : 45000

after call by reference: 40500

Explain why the call by value leaves the fee at 45000.

Alternative Concept and Complexity Analysis

If the concession function returned the new fee (long long applyConcession(long long fee, int percent)) instead of changing the fee through a reference, what would the solution be? Analyse the time (O(1) in all versions) and the space copied per call for a long long and for a large Student object passed by value, by reference and by const reference.

Evidence Expected

Three implementations, predictions before execution, observed results and a defended interface design.

ANS:

Problem: Hostel Fee Concession Updater

1. Three Methods of Passing Arguments

Call by Value: A copy of the original fee is passed to the function. Changes made inside the function do not affect the original fee.

Call by Reference: The function receives a reference to the original variable. Changes directly affect the caller's fee.

Call by Pointer: The address of the fee is passed to the function. The value is modified using the dereference operator *.

2. C++ Program

```
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
   cout << "Inside call by value: " << fee << endl;
}

void applyByReference(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}

void applyByPointer(long long* fee, int percent)
{
   *fee -= *fee * percent / 100;
}

int main()
{
   long long fee1 = 45000, fee2 = 45000, fee3 = 45000;

   applyByValue(fee1, 10);
   cout << "After call by value: " << fee1 << endl;

   applyByReference(fee2, 10);
   cout << "After call by reference: " << fee2 << endl;

   applyByPointer(&fee3, 10);
   cout << "After call by pointer: " << fee3 << endl;

   return 0;
}
```

3. Output

```
Inside call by value: 40500
After call by value: 45000
After call by reference: 40500
```

After call by pointer: 40500

4. Inquiry

- Call by value stores a separate copy of the parameter.
- Call by reference directly accesses the original variable.
- Call by pointer accesses the original variable through its memory address.
- A const reference is used when a function needs to access an object without modifying it, especially for large objects.

5. Design Decision

Call by reference is suitable for applying multiple concessions because the fee can be updated directly without creating additional copies. The function can apply each concession successively to the updated fee.

6. Predict the Output

My predicted output (write before running):

13 12 13
100 100
16 18 6

Explanation:

- change(a, b, &c) increases b and c by 10. The value of a remains unchanged because it is passed by value.
- bigger(a, c) = 100 returns a reference to the larger variable. Therefore, c becomes 100.
- show(1) uses default values 5 and 10, producing 16.
- show(1, 2) uses the default value 10 for z, producing 13.
- show(1, 2, 3) uses all supplied arguments, producing 6.

Correction to prediction: The exact output is:

1 12 13
100 100
16 13 6

7. Complete the Code

Completed Program:

```
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent)
```

```
{
   fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}

int main()
{
   long long hostelFee = 45000;

   applyByValue(hostelFee, 10);
   cout << "after call by value : " << hostelFee << '\n';

   applyConcession(hostelFee, 10);
   cout << "after call by reference: " << hostelFee << '\n';

   return 0;
}
```

Output:

```
after call by value : 45000
after call by reference: 40500
```

Explanation:

Call by value modifies only the local copy of the fee, so the original fee remains 45000. Call by reference modifies the original variable, reducing it to 40500.

8. Alternative Concept and Complexity Analysis

If the function returns the updated fee instead of modifying it through a reference, the implementation is:

```
long long applyConcession(long long fee, int percent)
{
   return fee - fee * percent / 100;
}

// Function call
```

hostelFee = applyConcession(hostelFee, 10);

Method	Time Complexity	Space Complexity
Call by value	O(1)	O(1)
Call by reference	O(1)	O(1)
Call by const reference	O(1)	O(1)

A long long requires a small fixed-size copy when passed by value. A large Student object requires copying all its data, whereas reference and const reference avoid copying the object.

Conclusion:

Call by reference is preferred for updating hostel fees because it directly modifies the original value without unnecessary copying. Call by value is useful when the original data must remain unchanged.

Q2 – Return by Reference and Its Dangers 4 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Examination Marks Register

The marks register keeps the marks of a class in an array. A function returns a reference to the highest mark so that a moderation of +2 can be applied directly to it. A second function returns a reference to a local variable.

Your Challenge

Implement both functions and show:

• How returning a reference lets the caller change the array element (and even use the call on the left of =)

• What the compiler warns about when a reference to a local variable is returned, and why using it is undefined behaviour

• The difference between returning a reference, returning a const reference and returning a copy

Changed Condition

The register is now stored in a std::vector that may grow. Explain why a reference obtained before the vector grows may become invalid.

Alternative Concept and Complexity Analysis

If the highest mark were returned as an index (int) or as a pointer (int*) instead of a reference (int&), what would the calling code look like? Analyse the time to find the highest mark (O(n) in all three) and the space (O(1)), and compare the safety of each when the array is later replaced by a growing vector.

Evidence Expected

Working functions, compiler-warning evidence, a correct explanation of dangling references and a justified response to the changed condition.

ANS:

Problem: Examination Marks Register

1. C++ Program

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int n)
{
   int maxIndex = 0;

   for (int i = 1; i < n; i++)
   {
      if (marks[i] > marks[maxIndex])
         maxIndex = i;
   }

   return marks[maxIndex];
}

// Returning a reference to a local variable (unsafe)
int& invalidReference()
{
   int x = 50;
   return x;
}

int main()
{
   int marks[] = {75, 89, 92, 84, 78};
   int n = 5;
```

```
    cout << "Highest mark: " << highestMark(marks, n) << endl;

    highestMark(marks, n) += 2;
    cout << "After moderation: " << highestMark(marks, n) << endl;

    highestMark(marks, n) = 100;
    cout << "After updating: " << highestMark(marks, n) << endl;

    return 0;
}
```

2. Output

Highest mark: 92

After moderation: 94

After updating: 100

3. Explanation

- Returning a reference allows the function to directly access and modify the original array element.
- highestMark(marks, n) += 2 increases the highest mark by 2.
- highestMark(marks, n) = 100 directly assigns a new value to that array element.
- Returning a reference to a local variable is unsafe because the local variable is destroyed when the function ends.

Compiler Warning:

The compiler may display a warning similar to:

warning: reference to local variable 'x' returned

Using this reference causes undefined behaviour because the referenced variable no longer exists after the function returns.

4. Difference Between Reference Types

Type	Description
int&	Allows modification of the original value.
const int&	Allows reading but prevents modification through the reference.
int	Returns a copy. Changes do not affect the original value.

5. Changed Condition – Using std::vector

When the marks register uses std::vector, adding new elements may cause reallocation. This moves the elements to a new memory location and invalidates previously obtained references, pointers and iterators.

Therefore, references should not be stored and reused after operations that may reallocate the vector.

6. Alternative Concept and Complexity Analysis

Returning an Index:

```
int highestIndex(int marks[], int n)
{
   int index = 0;

   for (int i = 1; i < n; i++)
      if (marks[i] > marks[index])
         index = i;

   return index;
}

// Usage
int index = highestIndex(marks, 5);
marks[index] += 2;
```

Returning a Pointer:

```
int* highestPointer(int marks[], int n)
{
   int index = 0;

   for (int i = 1; i < n; i++)
      if (marks[i] > marks[index])
         index = i;

   return &marks[index];
}

// Usage
*highestPointer(marks, 5) += 2;
```

Complexity Analysis:

Method	Time Complexity	Space Complexity
Return by reference	O(n)	O(1)
Return index	O(n)	O(1)
Return pointer	O(n)	O(1)

Comparison:

- Reference: Easy to use, but becomes invalid if the vector reallocates.
- Index: Safer when the vector grows because the index can be used again to access the element.
- Pointer: Directly accesses the element but becomes invalid after vector reallocation.

7. Conclusion

Returning by reference is useful for directly modifying array elements without copying them. However, returning references to local variables must be avoided because they cause undefined behaviour. For a growing vector, returning an index is a safer design because it can be used to access the element again after reallocation.

Q3 – Default Arguments and Function Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Courier Charge Calculator

A courier counter charges by weight; express delivery and insurance are optional. The developer wrote charge(double weight), charge(double weight, bool express) and charge(double weight, bool express = false, bool insured = false). The program no longer compiles.

Your Challenge

Explain the compiler error (ambiguous call). Then design the charge functions so

that: • Common calls are short (default arguments)

• Different kinds of input (weight in grams as int, weight in kg as double, a Parcel object) are handled by overloading

• No call is ambiguous

Before execution, predict which function each of six calls you design will select, including charge(2) and charge(2.0f).

Inquiry

Explain how the compiler chooses among overloaded functions (exact match, promotion, conversion), and why default arguments must be given from the right.

Alternative Concept and Complexity Analysis

If the courier charge took a single Parcel structure with optional fields (or a function template) instead of several overloaded functions with default arguments, what would the solution be? Analyse the time of overload resolution (at compile time, no run-time cost) and the space copied per call, and compare how each design handles a new option.

Evidence Expected

An explained ambiguity, a redesigned set of functions, predicted and observed selections, and a reasoned explanation of overload resolution.

ANS:

1. Compiler Error

The program produces an ambiguous function call because multiple overloaded functions can accept the same argument. Default arguments make it difficult for the compiler to distinguish between the functions.

2. Redesigned C++ Program

```
#include <iostream>
using namespace std;

struct Parcel
{
   double weight;
   bool express;
   bool insured;
};

void charge(int grams)
{
   cout << "Weight in grams: " << grams << endl;
}

void charge(double kg, bool express = false, bool insured = false)
{
   double cost = kg * 50;
```

```
    if (express) cost += 30;
    if (insured) cost += 20;

    cout << "Weight in kg: " << kg
         << ", Charge: " << cost << endl;
}

void charge(Parcel p)
{
    charge(p.weight, p.express, p.insured);
}

int main()
{
    charge(2);
    charge(2.0f);
    charge(2.5);
    charge(2.5, true);
    charge(2.5, true, true);

    Parcel p = {3.0, false, true};
    charge(p);

    return 0;
}
```

3. Predicted and Observed Output

My predicted output:

Weight in grams: 2

Weight in kg: 2, Charge: 100

Weight in kg: 2.5, Charge: 125

Weight in kg: 2.5, Charge: 155

Weight in kg: 2.5, Charge: 175

Weight in kg: 3, Charge: 170

Observed output:

Weight in grams: 2

Weight in kg: 2, Charge: 100

Weight in kg: 2.5, Charge: 125

Weight in kg: 2.5, Charge: 155

Weight in kg: 2.5, Charge: 175

Weight in kg: 3, Charge: 170

4. Function Selection

Function call	Selected function
charge(2)	charge(int)
charge(2.0f)	charge(double, bool, bool)
charge(2.5)	charge(double, bool, bool)
charge(2.5, true)	charge(double, bool, bool)
charge(2.5, true, true)	charge(double, bool, bool)
charge(p)	charge(Parcel)

5. Inquiry

The compiler selects an overloaded function based on the best match:

- Exact match: The argument type matches the parameter type.
- Promotion: A smaller type is promoted to a suitable larger type, such as float to double.
- Conversion: The compiler converts one type into another compatible type.

Default arguments must be specified from right to left because trailing parameters can be omitted during a function call.

6. Alternative Concept and Complexity Analysis

Instead of multiple overloaded functions, a single Parcel structure can store weight, express delivery and insurance details.

Example:

```
struct Parcel
{
   double weight;
   bool express = false;
```

```
    bool insured = false;
};
```

A single function can accept the Parcel object and calculate the charge using its fields.

Concept	Time Complexity	Space Complexity
Function overloading	O(1) runtime selection	Depends on arguments
Parcel structure	O(1)	O(1) for fixed fields
Function template	O(1) runtime selection	Depends on arguments

Function overloading provides separate functions for different input types, while a Parcel structure makes adding new options easier by adding fields.

7. Conclusion

Function overloading allows functions to handle different types of arguments. Default arguments reduce the number of parameters required in common calls. Combining both concepts carefully avoids ambiguity and makes the courier charge calculator easier to maintain.

Q4 – Specifying a Class with an Array Inside It 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Savings Account Mini-Statement

A bank wants each SavingsAccount object to remember its last 10 transactions for the mini statement printed at an ATM. Older transactions are discarded.

Your Challenge

Specify the class with private data (account number, balance, an array of 10 transactions and a count) and public member functions deposit(), withdraw() and miniStatement(). Define some member functions inside the class and some outside using the scope resolution operator.

Inquiry

Explain why the array must be private, what happens when the 11th transaction arrives if you shift the array versus use a circular index, and the cost of each.

Design Decision

The bank now wants the last 50 transactions. Decide which design changes least and defend it.

Alternative Concept and Complexity Analysis

If the last 10 transactions were kept in a std::deque (push_back and pop_front), or in a circular buffer with a start index, instead of shifting a fixed array, what would the solution be? Analyse the time to add a transaction (O(k) for shifting k elements versus O(1)) and the space used in each design.

Evidence Expected

A complete class specification, member functions defined inside and outside the class, a justified handling of the 11th transaction and a defended design.

ANS:

1. C++ Program

```
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount
{
private:
   int accountNumber;
   double balance;
   double transactions[10];
   int count;

public:
   SavingsAccount(int acc, double bal)
   {
      accountNumber = acc;
      balance = bal;
      count = 0;
   }

   void addTransaction(double amount);

   void deposit(double amount)
   {
      balance += amount;
      addTransaction(amount);
   }
```

```
    void withdraw(double amount)
    {
        if (amount <= balance)
        {
            balance -= amount;
            addTransaction(-amount);
        }
        else
            cout << "Insufficient balance" << endl;
    }

    void miniStatement();
};

void SavingsAccount::addTransaction(double amount)
{
    if (count < 10)
        transactions[count++] = amount;
    else
    {
        for (int i = 0; i < 9; i++)
            transactions[i] = transactions[i + 1];

        transactions[9] = amount;
    }
}

void SavingsAccount::miniStatement()
{
    cout << "Account Number: " << accountNumber << endl;
    cout << "Balance: " << balance << endl;
    cout << "Last Transactions:" << endl;

    for (int i = 0; i < count; i++)
        cout << transactions[i] << endl;
}

int main()
{
```

```
    SavingsAccount acc(101, 5000);

    acc.deposit(1000);
    acc.withdraw(500);
    acc.deposit(200);
    acc.withdraw(300);

    acc.miniStatement();

    return 0;
}
```

2. Output

Account Number: 101
Balance: 5400
Last Transactions:
1000
-500
200
-300

3. Inquiry

- The transaction array must be private to protect account information and prevent direct modification from outside the class.
- When the 11th transaction arrives, the oldest transaction is removed.
- In the shifting method, all remaining transactions are shifted one position, requiring O(k) time.
- In the circular index method, the oldest transaction is overwritten directly, requiring O(1) time.

4. Design Decision

For storing the last 50 transactions, I would choose a circular buffer. Only the array size and capacity need to be changed. It avoids shifting elements and provides efficient transaction insertion.

5. Alternative Concept and Complexity Analysis

Using std::deque:

```
#include <deque>
using namespace std;
```

```
deque<double> transactions;

void addTransaction(double amount)
{
   if (transactions.size() == 10)
      transactions.pop_front();

   transactions.push_back(amount);
}
```

Method	Insertion Time	Space Complexity
Fixed array with shifting	O(k)	O(k)
Circular buffer	O(1)	O(k)
std::deque	O(1) at ends	O(k)

Here, k represents the maximum number of stored transactions.

6. Conclusion

A class with a private transaction array provides data protection and maintains account details. A circular buffer is preferred because it stores the latest transactions efficiently without shifting existing elements. It is suitable for both 10 and 50 transaction records.

Q5 – Static Members and Arrays of Objects 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – College Bus Seat Tickets

The college bus has a fixed number of seats. Each BusTicket object must receive the next seat number, tickets beyond the capacity go to a waiting list, and the transport office must know how many seats have been issued without looking at any particular ticket.

Design Starting Point

You are given only the following principle:
Data that belongs to the whole class – not to any one object – should exist only once, however many objects are created.

Your Challenge

Convert this principle into a class design.

Determine:

• Which members must be static, and where a static data member must be defined • Why a static member function cannot use this or a non-static member • How an array of BusTicket objects is created and initialised

• What happens to the count when tickets are created inside a loop or a function

Test Your Design

Create an array of tickets larger than the capacity and verify the seat numbers and the waiting list.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

class Token {
 static int next;
 int id;

public:
 Token() { id = ++next; }
 int get() const { return id; }
 static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
 Token counter[3];
```

```
 Token extra;
 std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
<< ' ' << Token::issued() << '\n';
}
```

My predicted output (write before running):

Explain why the static counter starts at 100 and how many times the constructor ran. Complete the Code

Complete TODO 1 and TODO 2 below using the static members, and execute the program. #include <iostream>

#include <string>

```
class BusTicket {
 static int issued; // shared by all BusTicket objects
 static const int CAPACITY = 3;
 int seat;
 std::string name;

public:
 explicit BusTicket(const std::string& n) : seat(0), name(n)
 {
 // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
 // give this ticket the next seat number; otherwise leave seat as 0 (waiting  //
```

```
list)

 }

 static int count()
 {
 // TODO 2: Return the number of seats issued so far

 }

 void print() const
 {
 std::cout << name << ": "
 << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))  << '\n';
 }
};

int BusTicket::issued = 0; // definition of the static data member

int main()
{
 BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
BusTicket("Chitra"), BusTicket("Dev")};
 for (const BusTicket& t : trip) t.print();
 std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

```
Asha: Seat 1
```

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

Create the tickets inside a function that is called twice and explain how the count

behaves. Alternative Concept and Complexity Analysis

If the seat counter were a global variable, or were kept in a separate TicketOffice object passed to every BusTicket, instead of a static data member, what would the solution be? Analyse the time (O(1) in all) and the space (one counter in all), and compare the encapsulation and the ability to run two buses at once.

Evidence Expected

A student-derived class design using static members, an array of objects, test evidence and a reasoned explanation of static members.

ANS:

1. Static Members

Static data members are shared by all objects of a class. They store common information only once. A static member function can directly access only static members because it does not belong to a particular object and has no this pointer.

2. Predict the Output

```
#include <iostream>
using namespace std;

class Token {
   static int next;
   int id;
public:
   Token() { id = ++next; }
   int get() const { return id; }
   static int issued() { return next - 100; }
};

int Token::next = 100;
```

```
int main() {
    Token counter[3];
    Token extra;
    cout << counter[0].get() << ' ' << counter[2].get()
         << ' ' << extra.get() << ' ' << Token::issued() << '\n';
}
```

Output:

101 103 104 4

Explanation: The static variable next starts at 100. The constructor runs four times: three times for the array and once for extra. Therefore, the IDs are 101, 102, 103 and 104. The issued count is 104 − 100 = 4.

3. Completed BusTicket Program

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;

public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            issued++;
            seat = issued;
        }
    }

    static int count() {
        return issued;
    }

    void print() const {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting"))
```

```
            << '\n';
    }
};

int BusTicket::issued = 0;

int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };

    for (const BusTicket& t : trip)
        t.print();

    cout << "Issued: " << BusTicket::count() << '\n';
    return 0;
}
```

Output:

Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3

4. Creating Tickets Inside a Function

When tickets are created inside a function, the static variable retains its value even after the function ends. If the function is called twice, the count continues from its previous value because static data members are shared by all objects.

5. Alternative Design and Complexity Analysis

A global variable can also maintain the ticket count, but it is accessible throughout the program and reduces data protection. A separate TicketOffice object can maintain the count and be passed to each ticket. This provides better separation and supports multiple buses with separate counters.

Operation	Time Complexity

Create a ticket	O(1)
Assign a seat	O(1)
Read issued count	O(1)
Print all tickets	O(n)

Space complexity for the shared counter is O(1), excluding ticket objects. An array of n tickets requires O(n) space.

Result: Static members successfully maintain a common ticket count, while arrays of objects create multiple tickets. The program assigns seats within the bus capacity and places extra passengers on the waiting list.

Q6 – Friend Functions and Inline Functions 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Train Journey Planner

A journey planner stores departure and arrival times in a Time class whose hours and minutes are private. It must compute the minutes between two times and decide which time is earlier, and these small functions are called millions of times while planning connections.

Your Challenge

Design the comparison. Determine:

• Why a friend function is needed when the function works on two objects equally

• How a friend function differs from a member function in how it is called and what it can access

• Whether the small functions should be inline, and what inline actually asks the compiler to do

• What would be lost if the private members were made public instead

Correctness Check

Test with times on either side of noon and the same time twice, and verify by

hand. Growth Investigation

Measure the time for 10,000,000 calls with and without inline (compile with -O0 and with -O2), and explain the result.

Deployment Decision

Decide whether the friend functions should stay or be replaced by member functions and getters, and defend your decision.

Complete the Code

Complete TODO 1 and TODO 2 below inside the friend functions, and execute the

program. #include <iostream>

```
class Time {
 int h, m;

public:
 Time(int hh, int mm) : h(hh), m(mm) {}
 friend int minutesBetween(const Time& a, const Time& b);
 friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
 // TODO 1: Using the private members h and m, return the absolute difference between
// the two times in minutes

}

bool earlier(const Time& a, const Time& b)
{
 // TODO 2: Return true if a is earlier in the day than b

}
```

```
int main()
{
 Time departure(9, 45), arrival(13, 10);
 std::cout << minutesBetween(departure, arrival) << ' '
 << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output:

205 false

Test the same time twice and a pair of times on either side of noon.

Alternative Concept and Complexity Analysis

If the comparison were written as member functions (a.minutesTo(b), a.isEarlier(b)) or with public getters instead of friend functions, what would the solution be? Analyse the time per call (the same after inlining, O(1)) and measure it at -O0 and -O2, and compare the encapsulation of the three designs.

Evidence Expected

A justified friend-function design, correctness tests, measured inline timings at two optimisation levels and an evidence-based decision.

ANS:

1. Friend Functions

A friend function is a non-member function that is allowed to access the private and protected members of a class. It is useful when an operation works equally on two objects. Unlike a member function, it is called directly without using an object and does not have a this pointer.

2. Friend Function vs Member Function

- A friend function is called like minutesBetween(a, b).
- A member function is called using an object, such as a.minutesTo(b).
- A friend function can access private members of the class in which it is declared as a friend.
- A normal non-member function cannot directly access private members.

- A member function can directly access the private members of its own object.

3. Inline Functions

Inline functions are suitable for small, frequently called functions because they request the compiler to replace a function call with its function body where appropriate. This may reduce function-call overhead. However, the compiler can ignore the inline request and decide whether to inline the function.

4. Completed Program

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time {
   int h, m;

public:
   Time(int hh, int mm) : h(hh), m(mm) {}

   friend inline int minutesBetween(const Time& a, const Time& b);
   friend inline bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b) {
   int t1 = a.h * 60 + a.m;
   int t2 = b.h * 60 + b.m;
   return abs(t1 - t2);
}

inline bool earlier(const Time& a, const Time& b) {
   return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

int main() {
   Time departure(9, 45), arrival(13, 10);

   cout << minutesBetween(departure, arrival) << ' '
        << boolalpha << earlier(arrival, departure) << '\n';

   Time t1(11, 30), t2(12, 15);
```

```
    cout << minutesBetween(t1, t2) << ' '
         << earlier(t1, t2) << '\n';

    Time t3(10, 20), t4(10, 20);
    cout << minutesBetween(t3, t4) << ' '
         << earlier(t3, t4) << '\n';

    return 0;
}
```

Output:

205 false
45 true
0 false

5. Correctness Verification

- 9:45 to 13:10 = 205 minutes.
- 11:30 is earlier than 12:15, and the difference is 45 minutes.
- Two identical times have a difference of 0 minutes. Neither time is earlier than the other.

The program correctly handles times before and after noon and identical times.

6. Inline Performance Investigation

To measure performance, compile and run the program with different optimisation levels:

```
g++ -O0 main.cpp -o test_O0
./test_O0

g++ -O2 main.cpp -o test_O2
./test_O2
```

For a fair comparison, create two versions of the functions: one marked inline and another without inline. Use std::chrono to measure 10,000,000 calls and store the results in a checksum so the compiler cannot remove the calculations.

Optimisation	**Inline Version**	**Non-inline Version**
-O0	Record measured time	Record measured time

-O2	Record measured time	Record measured time

At -O0, function-call overhead may be noticeable. At -O2, the compiler may automatically inline suitable functions, so the difference can become smaller. Actual timings depend on the processor and compiler.

7. Alternative Design and Complexity Analysis

The same operations can be implemented using member functions:

```
int minutesTo(const Time& b) const {
   return abs((h * 60 + m) - (b.h * 60 + b.m));
}

bool isEarlier(const Time& b) const {
   return (h * 60 + m) < (b.h * 60 + b.m);
}
```

Another option is to provide public getter functions for hours and minutes. Normal non-member functions can then use the getters without accessing private members directly.

Design	Encapsulation
Friend functions	Direct access to private data, but friendship is limited to declared functions
Member functions	Strong encapsulation because operations remain inside the class
Public getters	Private data remains protected, but getter methods expose selected information

8. Deployment Decision

Friend functions are suitable because the comparison and difference operations use two Time objects equally. However, member functions provide better encapsulation by keeping related operations inside the class. For this journey planner, member functions are preferable if the operations are mainly performed by Time objects. Friend functions can be retained when non-member operations need direct access to private data.

Result: Friend functions successfully access private time values to calculate the difference and compare two times. Inline functions may reduce function-call overhead, while compiler optimisation determines the actual performance.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13 Marks

Bloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select ONE program that shows a significant limitation as the data grows or the requirement changes.

You are not permitted simply to replace your code with a library facility and claim it is better. You must understand the problem first and then develop an appropriate redesign.
A. Diagnose the Limitation – 2 Marks

Using your own evidence, identify where the design fails or slows down and which characteristic of the original design causes it.

Evidence Expected

A precise diagnosis supported by evidence from your original

program. B. Define the Design Objective – 2 Marks

State a measurable objective for the redesign.

Evidence Expected

A clearly stated objective derived from the identified limitation.

C. Design the Revised Solution – 4 Marks

Explain what you have changed in the class or function design, which C++ feature you use, whether extra memory is needed, and any new trade-off. Provide a class diagram or pseudocode.

Evidence Expected

A coherent redesign directly addressing the diagnosed limitation.

D. Predict Before Implementing – 1 Mark

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate the Redesign – 2 Marks

Compare Original → Redesigned using the same tests and

measurements. Evidence Expected

Comparable evidence for both versions.

F. Defend the Final Design – 2 Marks

Evaluate whether the objective was achieved, new costs, and where the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly explain why.

Evidence Expected

A critical, evidence-based evaluation and a defensible conclusion.

ANS:

A. Diagnose the Limitation

The original SavingsAccount program uses an array to store the last 10 transactions. When the array becomes full, the oldest transaction is removed by shifting all remaining elements one position to the left.

This requires multiple assignments whenever a new transaction is added. As the number of stored transactions increases, shifting becomes inefficient.

B. Define the Design Objective

The objective is to store the latest 10 transactions without shifting existing elements whenever a new transaction is added.

The redesigned solution should:

- Maintain only the latest 10 transactions.
- Add a new transaction in O(1) time.
- Display transactions in the correct order.
- Use a fixed amount of memory.

C. Design the Revised Solution

The redesigned solution uses a circular array with a next index and a count variable.

Changes made:

- The fixed-size array is retained.
- The next variable identifies the position for the next transaction.
- The count variable tracks the number of stored transactions.
- When the array becomes full, the oldest transaction is automatically overwritten.
- No shifting of elements is required.

Pseudocode:

```
Initialize transactions[10]
next = 0
count = 0

ADD TRANSACTION(amount):
   transactions[next] = amount
   next = (next + 1) % 10
   if count < 10:
      count = count + 1

DISPLAY TRANSACTIONS:
   start = (next - count + 10) % 10
   Repeat count times:
      Display transactions[start]
      start = (start + 1) % 10
```

D. Predict Before Implementing

The redesigned solution is expected to reduce the time required to add a transaction from O(n) to O(1), where n is the number of stored transactions. It will use the same fixed array size and maintain the latest 10 transactions.

E. Implement and Validate the Redesign

C++ Program:

```
#include <iostream>
using namespace std;

class SavingsAccount {
   double transactions[10];
   int next = 0;
   int count = 0;

public:
   void addTransaction(double amount) {
      transactions[next] = amount;
      next = (next + 1) % 10;

      if (count < 10)
         count++;
```

```
    }

    void miniStatement() const {
        int start = (next - count + 10) % 10;

        cout << "Last Transactions:\n";

        for (int i = 0; i < count; i++) {
            cout << transactions[start] << '\n';
            start = (start + 1) % 10;
        }
    }
};

int main() {
    SavingsAccount account;

    for (int i = 100; i <= 1200; i += 100)
        account.addTransaction(i);

    account.miniStatement();

    return 0;
}
```

Output:

```
Last Transactions:
300
400
500
600
700
800
900
1000
1100
1200
```

Comparison:

Feature	Original Array	Circular Array
Adding transaction when full	O(n)	O(1)
Display transactions	O(n)	O(n)
Memory usage	O(10)	O(10)
Shifting required	Yes	No
Latest transactions	10	10

The original and redesigned programs should be tested using the same 12 transactions. The output should contain the latest 10 transactions in both versions.

F. Defend the Final Design

The circular array achieves the objective by eliminating unnecessary shifting and allowing new transactions to be added in constant time. It uses the same fixed amount of memory as the original array.

The main trade-off is that circular indexing makes the implementation slightly more complex. The original array is easier to understand and is suitable when transactions are added infrequently.

Conclusion: The circular array is a better choice for frequently updated transaction records because it avoids shifting and provides O(1) insertion time while maintaining the latest 10 transactions.